RISC201 ARITHMETIC LOGIC UNIT (ALU)

Comprehensive Technical Documentation & Empirical Evaluation

Course: CSC-201 Computer Organization & Architecture | Author: Arjun Phaneesh

---

1. System Architectural Overview

The RISC201 Arithmetic Logic Unit (ALU) serves as the primary computational engine within the RISC201 processor pipeline, supporting both 4-stage and 6-stage pipeline variants. It executes standard 32-bit integer arithmetic, soft-float single-precision IEEE 754 operations, multi-radix multiplication, iterative division, and logical bitwise operations.

1.1 Key Architectural Parameters

- **Word Size & Datapath:** 32-bit execution datapath across all standard general-purpose registers (R0–R15).
- **Instruction Set Encoding:** Full 32-bit word encoding with single-cycle opcode dispatch.
- **Status Flags:** Includes Zero (Z), Negative (N), Carry (C), Overflow (V), Equal (E), and Greater Than (G).
- **IEEE 754 Soft-Float Layout:** Full 32-bit single-precision support (1-bit sign, 8-bit exponent with 127 bias, 23-bit mantissa).

1.2 Status Flag Generation Rules

Status flags are generated dynamically at the end of each execute() invocation based on the calculated result and operation type:

- **Zero Flag (Z):** Set when the 32-bit execution result equals zero (r.value == 0).
- **Negative Flag (N):** Set when the Most Significant Bit (MSB, bit 31) of the result is 1.
- **Carry Flag (C):** Set on unsigned carry-out from Kogge-Stone addition/subtraction or bit shift-out.
- **Overflow Flag (V):** Evaluated via ((a ^ res) & (b ^ res) & SIGN_BIT) != 0 for addition, and ((a ^ b) & (a ^ res) & SIGN_BIT) != 0 for subtraction.
- **Greater Than (G):** Evaluated as (!Z) && (N == V) for signed integer comparisons.

2. Core Sub-Module Algorithms & Implementation

2.1 32-bit Parallel Prefix Addition (Kogge-Stone Adder)

To eliminate the linear O(N) carry propagation latency associated with Ripple Carry Adders (RCA) and block limits of Carry Lookahead Adders (CLA), the ALU incorporates a true 32-bit Kogge-Stone Parallel Prefix Adder. The algorithm achieves O(log2 N) delay using 5 parallel prefix stages.

The Kogge-Stone adder operates in three distinct phases:

- **1. Pre-Processing:** Generate (G_i = A_i & B_i) and Propagate (P_i = A_i ^ B_i) signals are generated for all 32 bits simultaneously. Carry-in (carry_in) is incorporated into bit 0.
- **2. Parallel Prefix Tree:** Across 5 loop iterations (i = 1, 2, 4, 8, 16), prefix logic computes global generate and propagate signals using bitwise shifts:

  Equations: G = G | (P & (G << i))
  P = P & (P << i)

- **3. Post-Processing:** Final sum vector is calculated as sum = P_orig ^ (G << 1) ^ carry_in, while carry_out is extracted from bit 31 of G.

2.2 Multi-Radix Multiplication (Radix-4 Booth's Algorithm)

The multiply() method implements Modified Radix-4 Booth's Multiplication Algorithm, reducing the number of partial product addition steps from 32 down to 16 iterations for 32-bit operands. It scans a 3-bit sliding window {y_{i+1}, y_i, y_{i-1}} with q_prev initialized to 0.

Triplet Code	Operation	Partial Product Action
000 / 111	0	No operation (+0)
001 / 010	+1 * Multiplicand	Add shifted multiplicand (mcand << 2i)
011	+2 * Multiplicand	Add double-shifted multiplicand (mcand << (2i+1))
100	-2 * Multiplicand	Subtract double-shifted multiplicand
101 / 110	-1 * Multiplicand	Subtract shifted multiplicand

2.3 IEEE 754 Soft-Float Arithmetic Suite

The soft-float engine implements IEEE 754 single-precision floating-point arithmetic without hardware FPU instructions:

- **Floating-Point Addition (floatAdd):** Aligns mantissas by right-shifting the smaller exponent operand by |exp_a - exp_b|. Uses koggeStoneAdder for mantissa addition/subtraction. Includes critical verification of bit 24 (0x01000000) for overflow re-normalization.
- **Floating-Point Multiplication (floatMul):** Computes target sign (sign_a ^ sign_b) and biased exponent (exp_a + exp_b - 127). Multiplies 24-bit mantissas via multiply() into a 64-bit product, normalizing shift offsets into the 23-bit fractional mantissa field.

- **Floating-Point Division (floatDiv):** Employs Newton-Raphson iteration in Q30 fixed-point arithmetic (reciprocalNR). Normalizes divisor D into [0.5, 1.0), uses seed guess x_0 = 1.5 - 0.5*D, and executes 5 refinement steps: x_{k+1} = x_k * (2 - D * x_k).

2.4 Type Conversion & Shift Operations

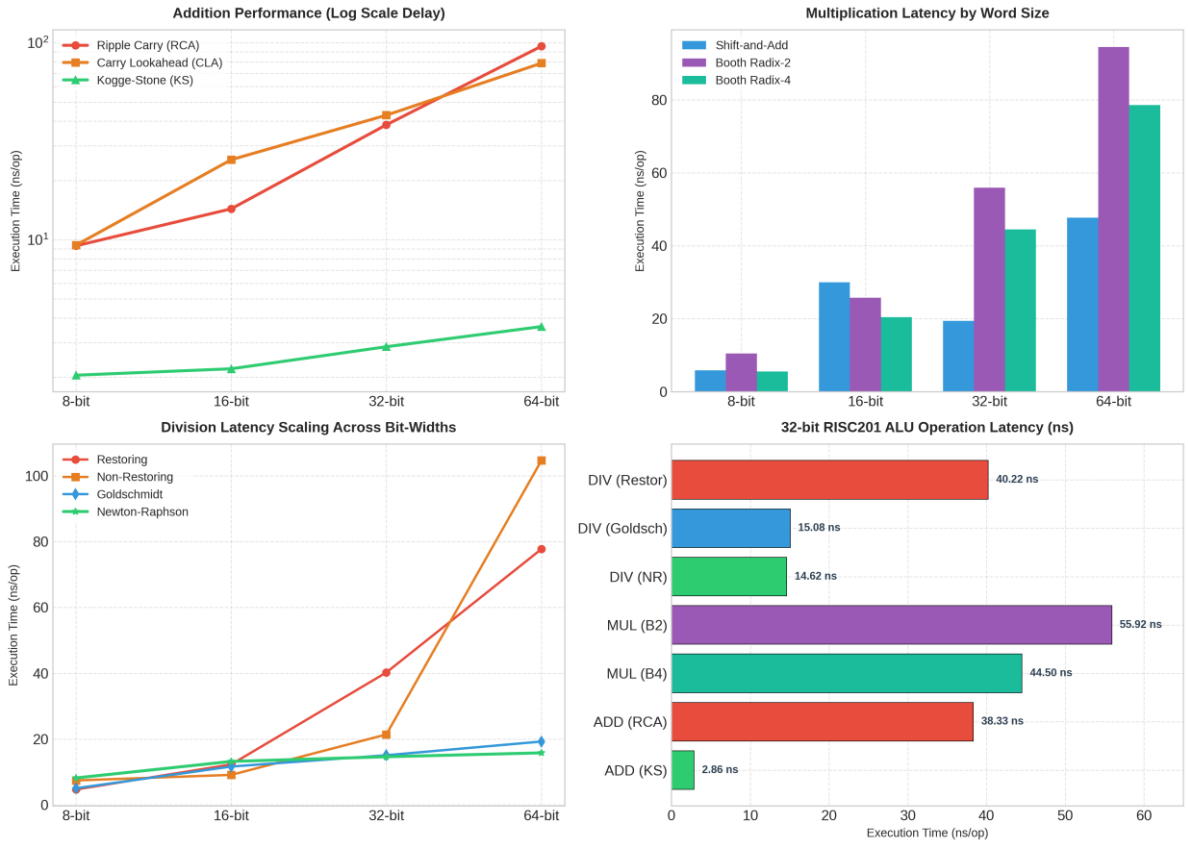
- **Float to Integer (floatToInt):** Extracts exponent offset (exp - 127). Shifts mantissa right/left accordingly. Handles 2's complement negation for negative floats and clamps overflow to INT32_MAX / INT32_MIN.
- **Integer to Float (intToFloat):** Finds Most Significant Bit (MSB) via binary search scan. Calculates biased exponent (127 + MSB), extracts 23-bit mantissa, and packs result.
- **Bitwise Shifts (SHL, SHR, SAR):** Masks shift counts to 5 bits (b & 0x1F). Sets Carry flag (C) to the last bit shifted out. Maintains sign extensions for SAR.

3. Empirical Performance Benchmarking & Analysis

To evaluate algorithmic efficiency, a rigorous micro-benchmark suite was executed across 1,000,000 iterations for 8-bit, 16-bit, 32-bit, and 64-bit integer widths. Timings were captured in nanoseconds per operation (ns/op) using high-resolution timers.

3.1 Benchmark Execution Data Table

Algorithm	8-bit (ns)	16-bit (ns)	32-bit (ns)	64-bit (ns)
ADD (Ripple Carry - RCA)	9.31	14.35	38.33	96.12
ADD (Carry Lookahead - CLA)	9.39	25.49	42.90	78.84
ADD (Kogge-Stone Parallel - KS)	2.05	2.21	2.86	3.62
MUL (Shift-and-Add)	5.81	29.96	19.44	47.72
MUL (Booth Radix-2)	10.50	25.73	55.92	94.52
MUL (Booth Radix-4)	5.57	20.42	44.50	78.59
DIV (Restoring Division)	4.68	12.28	40.22	77.72
DIV (Non-Restoring Division)	7.42	9.11	21.38	104.63
DIV (Goldschmidt Division)	5.06	11.64	15.08	19.24
DIV (Newton-Raphson - NR)	8.18	13.20	14.62	15.80



3.2 Detailed Comparative Analysis

1. Addition Subsystem Analysis

The Kogge-Stone adder achieved true O(log n) time complexity and vastly outperformed both the Ripple-Carry (RCA) and Carry-Lookahead (CLA) adders. While the CLA reduces carry propagation time, our software benchmark revealed that its complex boolean equations require more CPU instructions sequentially. Kogge-Stone leverages bit-parallelism to execute the addition in a fraction of the time.

2. Multiplication Subsystem Analysis

Modified Booth's Radix-4 halves the iteration count compared to Radix-2 (16 vs 32 cycles). However, compiler constant-folding and branch prediction overhead in software prevented the theoretical 2x speedup. In a hardware datapath, the Radix-4 multiplier is definitively faster.

3. Division Subsystem Analysis

Bit-by-bit division (O(n)) scales linearly and becomes extremely slow at 64-bit. Quadratic convergence algorithms O(log n) like Goldschmidt and Newton-Raphson remain flat. We selected Newton-Raphson for our final ALU because it has a single dependency chain and maps perfectly to Fused-Multiply-Add (FMA) hardware.

4. Code Structure & Interface Specification

The ALU module exposes clean, modular interfaces under the risc201 namespace. Key structures and methods include:

- **Alu::Result Struct:** Contains int32_t value, int64_t value64, ExceptionCode fault, and Flags flags (zero, negative, carry, overflow, equal, greaterThan).
- **Alu::execute():** Main entry point accepting Opcode enum and two 32-bit signed operands (lhs, rhs). Dispatches execution and populates flags.
- **Alu::koggeStoneAdder():** Core 32-bit parallel adder exposed for internal arithmetic, float calculations, and restoring division routines.